

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics

M2 Practical 8

Roll No.: T018	Name: Radhika Mali
Class: TYIT	Batch: 1
Date of Assignment: 11-9-26	Date/Time of Submission: 11-9-26

Part A: Confidence Interval Estimation

1. Load the supermarket_sales.csv dataset into a Pandas DataFrame.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
df = pd.read_csv("supermarket_sales.csv")
print(df.columns)
```

2. Display the first five records and check the size of the dataset.

Code:

```
print("First Five Records:")
print(df.head())
print("\nDataset Size:")
print(df.shape)
```

Output:

```
Index(['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
       'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Date',
       'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross income',
       'Rating'],
      dtype='str')
```

3. Select the revenue column for analysis.

Code:

```
revenue = df["Total"]
```

4. Check and handle missing values in the selected column.

Code:

```
print("\nMissing Values in Total:")
print(revenue.isnull().sum())
revenue = revenue.dropna()
```

Output:

```
Missing Values in Total:
0
```

5. Calculate the sample size, sample mean, and sample standard deviation.

Code:

```
n = len(revenue)
mean = revenue.mean()
std = revenue.std()
print("\nSample Size:", n)
print("Sample Mean:", mean)
print("Sample Standard Deviation:", std)
```

Function to calculate confidence interval

```
def confidence_interval(confidence):
    standard_error = std / np.sqrt(n)
    margin_error = stats.t.ppf(
        (1 + confidence) / 2,
        n - 1
    ) * standard_error
    lower = mean - margin_error
    upper = mean + margin_error
    return lower, upper
```

Output:

```
Sample Size: 1000
Sample Mean: 322.966749
Sample Standard Deviation: 245.88533510097187
```

6. Calculate the 90% confidence interval for the population mean revenue.

Code:

```
ci90 = confidence_interval(0.90)
```

7. Calculate the 95% confidence interval for the population mean revenue.

Code:

```
ci95 = confidence_interval(0.95)
```

8. Calculate the 99% confidence interval for the population mean revenue.

Code:

```
ci99 = confidence_interval(0.99)
```

9. Display the confidence intervals in a suitable table.

Code:

```
ci_table = pd.DataFrame({
    "Confidence Level": ["90%", "95%", "99%"],
    "Lower Bound": [ci90[0], ci95[0], ci99[0]],
    "Upper Bound": [ci90[1], ci95[1], ci99[1]]
})
print("\nConfidence Intervals:")
print(ci_table.round(2))
```

Output:

```
Confidence Intervals:
Confidence Level Lower Bound Upper Bound
0 90% 310.17 335.77
1 95% 307.71 338.23
2 99% 302.90 343.03
```

10. Compare the 90%, 95%, and 99% confidence intervals.**Code:**

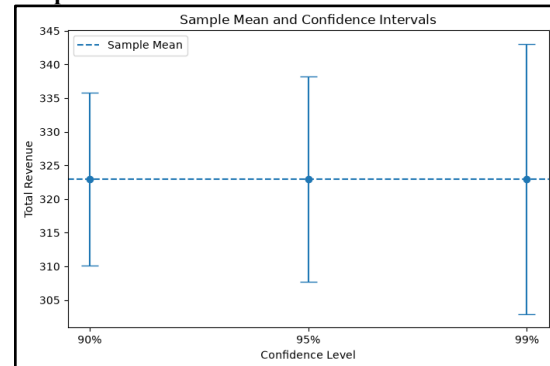
```
print("\nComparison:")
print("90% Confidence Interval:", ci90)
print("95% Confidence Interval:", ci95)
print("99% Confidence Interval:", ci99)
```

Output:

```
Comparison:
90% Confidence Interval: (np.float64(310.165191817398), np.float64(335.768306))
95% Confidence Interval: (np.float64(307.7084118264187), np.float64(338.22508))
99% Confidence Interval: (np.float64(302.8998536663743), np.float64(343.03364))
```

11. Create a suitable visualization to display the sample mean and confidence interval.**Code:**

```
confidence_levels = ["90%", "95%", "99%"]
lower_bounds = [
    ci90[0],
    ci95[0],
    ci99[0]
]
upper_bounds = [
    ci90[1],
    ci95[1],
    ci99[1]
]
errors_lower = [
    mean - x for x in lower_bounds
]
errors_upper = [
    x - mean for x in upper_bounds
]
plt.figure(figsize=(8, 5))
plt.errorbar(
    confidence_levels,
    [mean, mean, mean],
    yerr=[errors_lower, errors_upper],
    fmt='o',
    capsize=8
)
plt.axhline(
    mean,
    linestyle='--',
    label="Sample Mean"
)
plt.xlabel("Confidence Level")
plt.ylabel("Total Revenue")
plt.title("Sample Mean and Confidence Intervals")
plt.legend()
plt.show()
```

Output:**12. Interpret the 95% confidence interval in the context of supermarket revenue.****Code:**

```
print("\nInterpretation of 95% Confidence Interval:")
print(
    f"We are 95% confident that the true population "
    f"mean supermarket revenue lies between "
    f"{ci95[0]:.2f} and {ci95[1]:.2f}."
)
```

Output:

```
Interpretation of 95% Confidence Interval:
We are 95% confident that the true population mean supermarket revenue lies between 307.71 and 338.23.
```

13. Select the customer_type column and calculate the proportion of customers belonging to each category.**Code:**

```
customer_type = df["Customer type"]
print("\nCustomer Type Counts:")
print(customer_type.value_counts())
print("\nCustomer Type Proportions:")
proportions =
customer_type.value_counts(normalize=True)
print(proportions.round(4))
```

Output:

```
Customer Type Counts:
Customer type
Member    501
Normal    499
Name: count, dtype: int64

Customer Type Proportions:
Customer type
Member    0.501
Normal    0.499
Name: proportion, dtype: float64
```

14. Calculate the 95% confidence interval for the proportion of Member customers.**Code:**

```

member_count = (customer_type == "Member").sum()
total_customers = customer_type.notna().sum()
member_proportion = member_count / total_customers
z = stats.norm.ppf(0.975)
margin = z * np.sqrt(
    member_proportion *
    (1 - member_proportion) /
    total_customers
)
prop_lower = member_proportion - margin
prop_upper = member_proportion + margin
print("\nMember Proportion:")
print(member_proportion)
print("\n95% Confidence Interval for Member Proportion:")
print("Lower Bound:", prop_lower)
print("Upper Bound:", prop_upper)

```

Output:

```

Member Proportion:
0.501

95% Confidence Interval for Member Proportion:
Lower Bound: 0.47001031036433716
Upper Bound: 0.5319896896356628

```

15. Interpret the confidence interval for the population proportion in a business context.**Code:**

```

print("\nBusiness Interpretation:")
print(
    f"We are 95% confident that the true population "
    f"proportion of Member customers is between "
    f"{prop_lower * 100:.2f}% and "
    f"{prop_upper * 100:.2f}%."
)

```

Output:

```

Business Interpretation:
We are 95% confident that the true population proportion of Member customers

```

Part B: Bootstrapping:**16. Select the revenue column and calculate its original sample median.****Code & Output:**

```

original_median = revenue.median()
print("Original Sample Median:", original_median)

```

Original Sample Median: 253.848

17. Create a bootstrap sample by randomly selecting observations from the original sample with replacement.**Code & Output:**

```

bootstrap_sample = np.random.choice(
    revenue,
    size=len(revenue),
    replace=True
)
print("Bootstrap Sample:")
print(bootstrap_sample)

```

```

Bootstrap Sample:
[ 936.6   203.553  671.79   203.931  690.018  419.832  68.985
 160.86  429.1665  326.256  728.112   76.755  757.365  148.995
 391.419  474.348  411.3795  335.013  28.4235  135.576  418.95
 296.94   78.0045  225.2775  224.4375  181.44  132.762  181.44]

```

18. Calculate the median of the bootstrap sample.**Code & Output:**

```

bootstrap_median = np.median(bootstrap_sample)
print("Median of Bootstrap Sample:", bootstrap_median)

```

Median of Bootstrap Sample: 241.458

19. Repeat the bootstrapping process 1,000 times and store the median from each bootstrap sample.**Code & Output:**

```

bootstrap_medians = []
for i in range(1000):
    sample = np.random.choice(
        revenue,
        size=len(revenue),
        replace=True
    )
    bootstrap_medians.append(np.median(sample))
print("Total Bootstrap Medians:", len(bootstrap_medians))

```

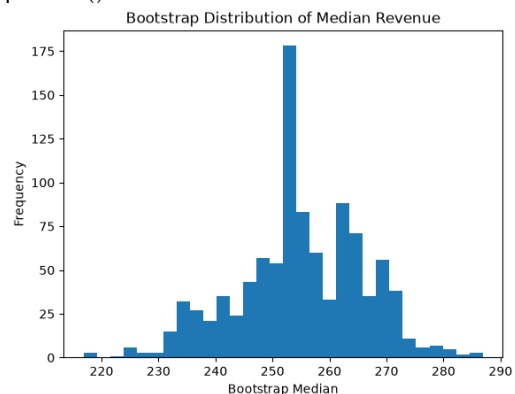
Total Bootstrap Medians: 1000

20. Create a histogram of the bootstrap median values.**Code & Output:**

```

plt.hist(bootstrap_medians, bins=30)
plt.xlabel("Bootstrap Median")
plt.ylabel("Frequency")
plt.title("Bootstrap Distribution of Median Revenue")
plt.show()

```



21. Calculate the 2.5th percentile and 97.5th percentile of the bootstrap median values.

Code & Output:

```
lower = np.percentile(bootstrap_medians, 2.5)
upper = np.percentile(bootstrap_medians, 97.5)
print("2.5th Percentile:", lower)
print("97.5th Percentile:", upper)
```

```
2.5th Percentile: 232.869
97.5th Percentile: 273.61372500000004
```

22. Use these percentile values to calculate the 95% bootstrap confidence interval for the population median.

Code & Output:

```
print("95% Bootstrap Confidence Interval:")
print("Lower:", lower)
print("Upper:", upper)
```

```
95% Bootstrap Confidence Interval:
Lower: 232.869
Upper: 273.61372500000004
```

23. Display the original sample median and the bootstrap confidence interval.

Code & Output:

```
print("\nOriginal Sample Median:", original_median)
print("95% Confidence Interval:", (lower, upper))
```

```
Original Sample Median: 253.848
95% Confidence Interval: (np.float64(232.869), np.float64(273.61372500000004))
```

24. Compare the original median with the bootstrap confidence interval.

Code & Output:

```
if lower <= original_median <= upper:
    print("The original median lies within the confidence interval.")
else:
    print("The original median lies outside the confidence interval.")
```

```
The original median lies within the confidence interval.
```

25. Interpret the bootstrap confidence interval in the context of supermarket revenue.

Code & Output:

```
print("\nInterpretation:")
print(
    "The population median supermarket revenue is reasonably "
    "estimated to lie between", lower, "and", upper
)
```

```
Interpretation:
The population median supermarket revenue is reasonably estimated to lie between 232.869 and 273.61372500000004
```

26. Write a conclusion explaining how confidence intervals and bootstrapping can help businesses make decisions using sample data.

Explanation : Bootstrapping helps businesses estimate population values from sample data. Confidence intervals show the possible range of the actual value and the uncertainty in the estimate. This helps businesses make better and more reliable decisions about revenue, sales, and business performance.